# Chapter 3

# METHODOLOGY

# TABLE OF CONTENTS


Conceptual Framework 5

System Environment Specifications 7

Section 1: Dataset 10

- 1.1 What to Gather 10
- 1.2 How to Gather 12
- 1.4 Why 14
- 1.5 Participant Screening Criteria 16
- 1.6 Task Design 17

Section 2: Procedure / Experiment 19

- 2.1 System Design and Implementation 19
- 2.2 Data Pre-processing 22
- 2.3 Validation 23
- 2.4 Study Session Procedure 24
- 2.5 Ethical Considerations 25

Section 3: System Testing 27

- 3.1 Evaluation Dataset 27
- 3.2 Evaluation Profile 27
- 3.3 Criteria and Formula 28
- 3.4 Data Analysis Plan 30
- 3.5 Threats to Validity 31

References 33

Appendices 35

## LIST OF TABLES

Table 3.1: Study evaluation machines 7

Table 3.2: SynapseOS minimum deployment requirements 9

# LIST OF FIGURES

Figure 3.1: Conceptual framework of SynapseOS 7

This chapter describes the research methodology employed in designing, implementing, and evaluating SynapseOS — a system that replaces the conventional graphical session layer with a conversational interface layer. The methodology is organized into three sections: Dataset, which defines the data to be collected and how it will be gathered; Procedure, which describes the step-by-step process of building the system and conducting the study; and System Testing, which defines the evaluation design, metrics, and statistical approach used to compare SynapseOS against the conventional Linux desktop.

This methodology directly answers the three research questions posed in Chapter 1: *RQ1*, how a conversational session layer can be designed as the primary interface to a Linux-based personal computing environment; *RQ2*, what natural-language-to-bash intent parsing and execution design — incorporating a reversibility-based confirmation gate and an undo mechanism — most effectively grounds natural language intent into safe, recoverable system actions; and *RQ3*, how SynapseOS compares to each participant's own primary operating system in task completion time, error rate, and user satisfaction across novice and power user populations. Section 1 defines the data gathered to answer RQ2 and RQ3; Section 2 describes the system built to answer RQ1 and RQ2; and Section 3 defines the evaluation design used to answer all three.

## Conceptual Framework

SynapseOS operates as a conversational shell layer positioned between the user and the Linux command-line environment. The user issues natural language commands through a persistent chat

interface. An intent parsing pipeline — powered by a locally-hosted small language model — translates each utterance into a shell command, which is executed in a bash subprocess. Output is captured and rendered back in the conversational interface. The user issues intent in natural language and receives results in natural language; the shell command is an implementation detail invisible to the user. This architecture is illustrated in Figure 3.1.

The framework is evaluated through a controlled user study in which participants complete a fixed set of tasks under two conditions: using SynapseOS as the interface, and using their own primary operating system — Windows, macOS, or Linux — as the baseline. This expert baseline design pits SynapseOS against the environment each participant already knows and uses daily, generating empirical data on task completion time, error rate, and user satisfaction across the real-world OS landscape [17].

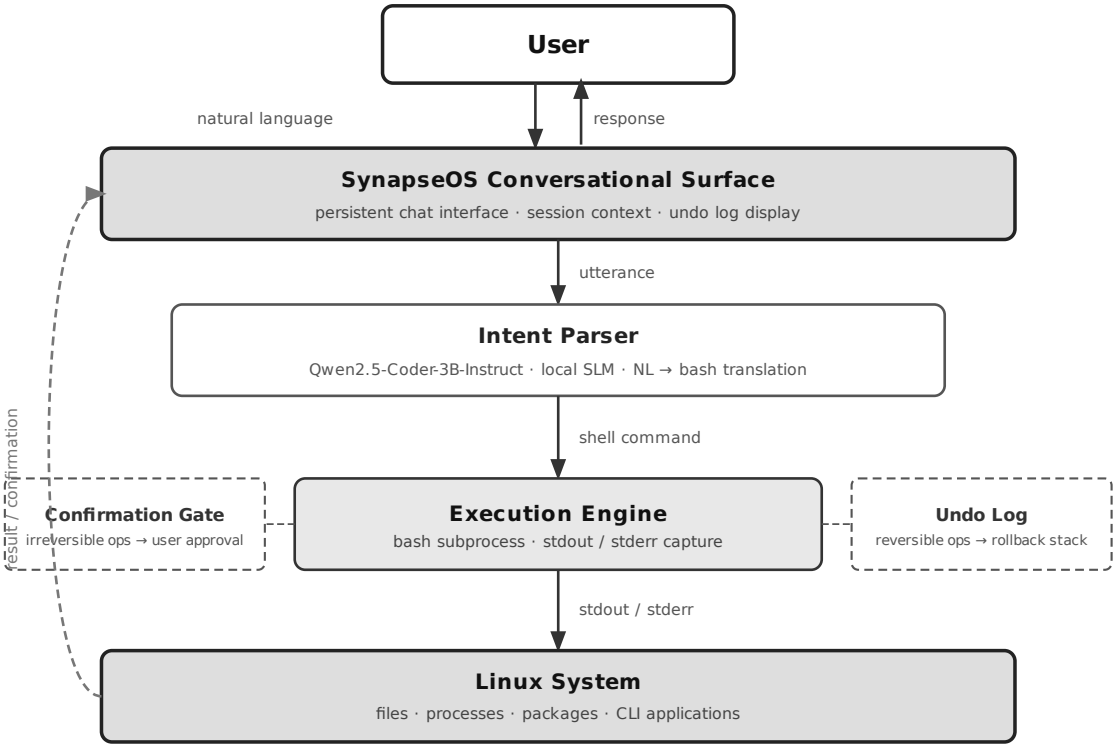

**Figure 3.1.** Conceptual framework of SynapseOS. The user issues a natural language command to the conversational surface, which routes the utterance to the intent parser (Qwen2.5-Coder-3B-Instruct, local SLM). The intent parser produces a shell command dispatched to the execution engine (bash subprocess). Irreversible operations are intercepted by the confirmation gate; reversible operations are logged to the undo stack. Command output (stdout/stderr) is captured, rendered in the conversational surface, and presented back to the user — completing the interaction loop.

## System Environment Specifications

The user study is conducted across three dedicated machines — one running SynapseOS (Condition A) and two covering the three supported baseline operating systems (Condition B). Table 3.1 describes each machine's role and operating environment. Exact processor, RAM, and storage specifications for each machine are confirmed and documented in the study protocol prior to the pilot study. Table 3.2 specifies the minimum hardware and software requirements for deploying SynapseOS independently of the study configuration.

**Table 3.1.** Study evaluation machines

| Machine | Role | OS | Notes |
|---|---|---|---|
| SynapseOS evaluation machine | Condition A — all participants | SynapseOS | GUI mode; fullscreen conversational interface running within an XFCE (X11 session) |

| | | | |
|---|---|---|---|
| | | | desktop environment; based on Debian 13 (Trixie); Ollama + Qwen2.5-Coder-3B-Instruct running locally |
| Windows / Linux dual-boot baseline machine | Condition B — Windows- or Linux-primary participants | Windows 11 or Ubuntu 24.04 LTS (GNOME), selected at boot | Standard consumer x86-64 hardware; Windows 11 and Ubuntu installed on separate partitions; standard user account per OS; no custom software. Facilitator boots to the OS matching the participant's assignment before the session begins. |
| macOS baseline machine | Condition B — macOS-primary participants | macOS (current release) | Apple Silicon hardware; standard user account |

**Table 3.2.** SynapseOS minimum deployment requirements. SynapseOS is distributed as a Linux system based on Debian 13 (Trixie).

| **Component** | **Minimum Requirement** |
|---|---|

| | |
|---|---|
| Processor | 64-bit x86 processor, 2-core minimum; no GPU required |
| RAM | 4 GB (TUI mode[1]); 8 GB recommended (GUI mode) |
| Storage | 10 GB available (operating system, language model, and application) |
| Display | TUI mode: any terminal or remote connection; no graphical environment required. GUI mode: graphical desktop environment required. |
| Language Model | Locally-hosted small language model (~1.8 GB)[2]; CPU-only inference; no GPU required. Cloud language model backend available as an optional, user-configured alternative. |
| Inference Server | Local inference server (Ollama)[3]; manages model lifecycle, loading, and query serving on the host machine. |

The user study evaluates SynapseOS in GUI mode, which requires a graphical desktop environment and 8 GB RAM. SynapseOS additionally supports a TUI mode — a terminal-based interface requiring no display server, running on as little as 2 vCPU and 4 GB RAM — designed for server and remote deployment environments accessible over SSH. TUI mode is not evaluated in the user study.

## Section 1: Dataset

The data gathered for this study falls into three categories: benchmark task data used to evaluate the system in a standardized environment, user study data collected from human participants in a controlled setting, and system performance data captured automatically through built-in telemetry during all sessions.

## 1.1 What to Gather

a) Benchmark Task Suite

The primary benchmark dataset is the OSWorld task suite [17], which provides standardized real-computer tasks across shell operations, file management, web browsing, and application-level interactions. OSWorld is the only existing benchmark that evaluates computer-using agents in real operating system environments, making it the most appropriate external standard for SynapseOS evaluation. Tasks are presented as natural language instructions; agents interact with a real virtual machine environment, and outcomes are evaluated by automated inspection of the resulting system state — whether the goal was actually achieved — rather than by comparing individual actions. In addition to OSWorld tasks, a custom cross-platform task suite is developed for this study, covering file search and organization, system and process monitoring, application and package management, and text and data processing — each task defined as a human-level goal achievable via SynapseOS natural language commands and via native OS tools on the comparison machine. Each task in both suites is documented with a description, expected outcome, task category, and difficulty level.

b) User Study Data

The following data types are collected from each participant during the user study:

- **Task completion time:** the elapsed time from when the task prompt is presented to when the participant signals task completion or the session logs a terminal event
- **Error rate:** the count of failed tasks, incorrect outcomes, and recovery attempts per participant per condition
- **User satisfaction scores:** post-study responses to the System Usability Scale (SUS) and the NASA Task Load Index (NASA-TLX), both validated instruments for measuring perceived usability and cognitive workload respectively
- **Qualitative feedback:** responses gathered through a semi-structured post-study interview covering perceived ease of use, trust in the system, and comparison between the two conditions

c) System Performance Data

The following telemetry data is captured automatically by SynapseOS throughout all sessions:

- Command execution latency: time elapsed from utterance submission to bash subprocess completion
- Undo log events: count and type of operations logged to the undo stack
- Confirmation prompts triggered: count of irreversible operations that required user confirmation
- Conversational turns per task: the number of utterances required to complete each task

- Intent parsing accuracy: whether the system correctly identified the user's intent on the first attempt

### 1.2 How to Gather

a) Benchmark Tasks

SynapseOS is deployed on the Debian 13 (Trixie) environment matching the OSWorld evaluation setup [17]. Each benchmark task is executed through the conversational interface, and results are captured programmatically through the built-in telemetry system. Tasks are run in automated sessions where SynapseOS is given the task prompt as a natural language instruction and the system's output is compared against the expected outcome defined in the benchmark. OSWorld evaluates autonomous agents by inspecting the resulting system state after independent task execution, not a human operator's performance; SynapseOS is the study's only autonomous agent, so it is the only system evaluated against this benchmark. Condition B has no autonomous-agent equivalent — a human participant operates it directly — and is instead evaluated through the custom cross-platform task suite administered in the user study (Section 1.2b).

b) User Study

The user study is conducted in a controlled laboratory setting with two machines: the SynapseOS evaluation machine (Debian 13, conversational interface) and the baseline machine running the participant's native OS — Windows 11, macOS, or Linux with a GNOME desktop — as determined during screening. Each participant

completes the full task set on both machines under a within-subjects design, with condition order counterbalanced across participants to mitigate learning and carryover effects. Specifically, half of the participants complete the SynapseOS condition first, and the remaining half complete the native OS condition first. Screen capture software records activity on both machines; the SynapseOS session logger supplements the recording for the SynapseOS condition.

The study involves 20 participants divided into two groups:

- **Novice users (n = 10):** participants with limited computing experience, capable only of basic tasks (web browsing, document editing) on any OS, and no prior command-line familiarity, recruited from the general university population
- **Power users (n = 10):** participants with demonstrated proficiency in computing on their primary OS — capable of performing file management, system monitoring, and process administration tasks efficiently — recruited from the university's information technology and computer science programs

Participants are recruited through university channels. Recruitment enforces a minimum quota of two participants per primary OS background (Windows, macOS, Linux), so that the realized sample cannot end up concentrated in a single OS and leave the per-OS exploratory subgroup analysis (Section 3.2) without data for the others. Informed consent is obtained from all participants prior to the session. Ethics approval is secured before recruitment begins. Participant data is anonymized at the point of collection.

After completing each condition, participants fill out the SUS and NASA-TLX questionnaires. After completing both conditions, a brief semi-structured interview is conducted to gather qualitative

feedback.

### 1.4 Why

The OSWorld benchmark is used as the primary evaluation standard because it is the only existing benchmark that evaluates computer-using agents in real operating system environments with real applications [17]. Its use allows SynapseOS results to be situated within the existing literature.

A custom cross-platform task suite is necessary because OSWorld's task coverage does not represent the human-goal-framed system tasks relevant to this study — file organization, system monitoring, process management, and text processing — achievable both through SynapseOS natural language commands and through participants' native OS tools (Windows, macOS, or Linux graphical desktop). Tasks are scoped to operations achievable via bash on SynapseOS and via native graphical or terminal tools on the comparison machine.

The evaluation follows a convergent (parallel) mixed-methods design: the quantitative performance measures — task completion time, error rate, SUS, and NASA-TLX — and the qualitative interview data are collected concurrently within each session and integrated at the interpretation stage, so that the measured differences between conditions and participants' accounts of why those differences arise inform one another rather than being read in isolation.

The within-subjects design is chosen because it allows each participant to serve as their own control, producing a direct comparison between conditions while requiring a smaller participant pool than a between-subjects design. Counterbalancing controls for

the primary threat to internal validity: the learning effect, wherein participants may perform better in the second condition simply due to familiarity with the tasks.

The expert baseline design — using each participant's primary OS as Condition B rather than a fixed Linux graphical desktop — is chosen because a fixed Linux baseline would confound interface quality with OS familiarity. The overwhelming majority of participants have never used a Linux desktop; performance under that condition would reflect unfamiliarity with a foreign environment, not a meaningful comparison of interface paradigms. The stronger and more ecologically valid test is whether SynapseOS can compete with what participants already know and use daily. A positive result against a participant's native environment is a more compelling argument for SynapseOS than a win against a system participants are encountering for the first time.

The dual-population design — novice and power users — directly addresses the gap identified in Chapter 1, where no prior work has evaluated a conversational interface across user populations with differing technical backgrounds [17]. The three data categories align with the three research questions: RQ1 is addressed by the system design and benchmark task coverage; RQ2 is addressed by system performance telemetry (command execution latency, intent parsing accuracy, undo and confirmation events); and RQ3 is addressed by the user study metrics (task completion time, error rate, satisfaction scores).

### 1.5 Participant Screening Criteria

Participants are assigned to the novice or power user group based on a pre-study screening questionnaire administered before the session begins. The questionnaire assesses the following dimensions:

- Years of general computer use
- Primary OS — the operating system used as the primary daily computing environment (Windows / macOS / Linux); must average four or more hours of use per day to qualify
- Self-reported proficiency on primary OS (basic / intermediate / advanced)
- Self-reported command-line familiarity (none / basic / intermediate / advanced)
- Prior exposure to conversational AI interfaces — frequency of use (never / occasionally / regularly / daily) and which tools (chatbots, voice assistants, code-completion or agentic coding assistants, other LLM-based tools)

Self-reported proficiency is confirmed with a short behavioral screener: three timed mini-tasks performed unaided on the participant's primary OS using its native tools, one per system-task category used in the main study — file organization (locate and relocate a set of recently modified files), process monitoring (identify the application consuming the most memory via the OS's native system monitor), and application management (determine whether a named application is installed and identify its version). Each task is scored pass/fail by the facilitator against a fixed time limit and completion criterion. The behavioral screener exists because self-reported proficiency alone is not a reliable classifier — a participant's technical background in one domain (e.g., software development) does not guarantee fluency with a given OS's native file management,

process monitoring, and system administration tools, which is the specific competency this study's novice/power-user split is intended to capture.

Participants are classified as **novice users** if they report basic proficiency on their primary OS, computing use limited to web browsing and document editing, no prior command-line experience on any platform, and fail the behavioral screener. Participants are classified as **power users** if they report advanced proficiency on their primary OS — capable of performing file management, process monitoring, and system administration tasks efficiently using that OS's native tools — regardless of which OS that is, and pass all three behavioral screener tasks. Participants whose self-report and behavioral screener results disagree, or who do not clearly satisfy either classification, are excluded. Primary OS is recorded for all participants as a grouping variable, and prior conversational AI exposure as a covariate, in the analysis; prior AI exposure is analyzed as an exploratory covariate only (see Section 3.2) rather than a primary grouping variable, since splitting the n = 20 sample along a third axis would leave subgroups too small to support a hypothesis test.

### 1.6 Task Design

The custom cross-platform task suite is designed to cover human-level system tasks achievable both through SynapseOS natural language commands and through native tools on the participant's primary OS. Task design follows four principles:

1. **Ecological validity** — tasks reflect actions a real user would perform in their everyday computing environment: finding and organizing files, monitoring system resources, managing processes, and handling text data
2. **Cross-platform equivalence** — all tasks are defined as human-level goals achievable both through SynapseOS natural language commands (mapped to bash) and through native tools on the participant's primary OS (File Explorer, Task Manager, Finder, Activity Monitor, GNOME, or equivalent). The method of achievement differs by platform; the human goal does not
3. **Difficulty stratification** — tasks are rated at three difficulty levels (simple, moderate, complex) based on the number of steps required and the specificity of the target operation
4. **Population appropriateness** — both participant groups receive the same task set, with task wording written in plain language accessible to novice users and free of OS-specific or CLI-specific terminology

The custom task suite comprises tasks organized into four categories: file search and organization, system and process monitoring, application and package management, and text and data processing. All tasks are defined at the human-goal level — achievable via SynapseOS natural language on the evaluation machine and via native graphical or terminal tools on the comparison machine. Each task is documented with a unique identifier, a natural language prompt as presented to the participant, the expected outcome, and the assigned difficulty level. Tasks are validated during the pilot study and revised as necessary before the main study begins.

## Section 2: Procedure / Experiment

Building SynapseOS and running the study that evaluates it are two related but distinct efforts. The system itself — the conversational interface layer and execution pipeline — is designed and implemented first, followed by the scaffolding that makes the resulting study trustworthy: collected data is processed and anonymized, the system and instruments are validated, each participant session follows a standardized procedure, and participant welfare is protected throughout.

### 2.1 System Design and Implementation

The construction of SynapseOS proceeds across two primary workstreams — the conversational interface layer and the execution pipeline — integrated into a unified session manager that launches in place of the conventional Linux graphical desktop.

a) Conversational Interface Layer

The conversational surface is implemented as a persistent, full-screen chat interface that serves as the primary and sole user-facing interface when SynapseOS is active as the session manager. The interface accepts natural language input, displays system responses and command output, and maintains conversational context across the session. An intent parsing pipeline translates each user utterance into a shell command for execution.

Conversational context is session-scoped: history is held in memory for the duration of the login session and is cleared on logout or reboot, with no persistence layer in the thesis prototype. When accumulated history approaches 75% of the intent parsing model's

8,000-token context limit, a rolling window discards the oldest turns to keep the conversation within budget. Verbose command output — such as a full directory listing or search result — is truncated to a compact summary before being appended to history, preventing a single command's output from consuming a disproportionate share of the available context.

SynapseOS is designed around a locally-hosted small language model (SLM) as the default, model-agnostic inference backend, with cloud inference available only as an explicit opt-in. Because the system operates at the OS level — processing every natural language command and observing terminal output, file paths, command history, and interaction patterns — routing inference through a cloud API would give an external provider continuous visibility into the user's computing activity. On-device inference eliminates this exposure by default; a user may still supply an API key for a supported cloud provider to substitute a hosted frontier model, entirely at their discretion. The default configuration requires no API key, no internet connection, and no per-query cost, making SynapseOS fully operational in offline and air-gapped environments.

The primary SLM is Qwen2.5-Coder-3B-Instruct, an open-weights[4] code-specialized model that demonstrated the strongest natural language-to-bash translation accuracy among models below 7B parameters in controlled evaluation [2]. Among the models directly compared in that evaluation, its code-specialized pretraining produced a clear margin over general-purpose models of comparable or larger size — 58% NL2Bash accuracy with prompt engineering, versus 49% for Llama 3.2-3B and 37% for a LoRA-fine-tuned Llama 3.2-1B — attributable to Qwen2.5-Coder's training corpus of 5.5 trillion code tokens against Llama 3.2's general-purpose pretraining.

At 4-bit quantization[5], the model requires approximately 1.8 GB of memory and runs on the target Debian 13 (Trixie) hardware without a dedicated GPU. The model is fine-tuned via Low-Rank Adaptation (LoRA)[6] on the NL2Bash corpus[7] and a custom SynapseOS task suite to maximize intent parsing accuracy.

Accuracy is measured using an execution-based scoring methodology rather than syntax-level string matching, following Westenfelder et al. [2] — see Section 2.3 for the full evaluation procedure.

This selection was reconfirmed against models released after the initial literature review, to guard against the fast-moving small-model landscape becoming stale before study execution; no smaller alternative offered comparable shell-generation evidence within the same CPU-only hardware budget [25]. Newer mixture-of-experts variants require all experts resident in memory regardless of active parameters per token, incompatible with this study's memory budget, and newer general-purpose small models report reasoning benchmarks but no shell-command generation evidence.

b) Execution Pipeline

Once the intent parser produces a shell command, SynapseOS passes it through two safety mechanisms before execution, following the reversibility-and-guardrail framing established for LLM-powered natural language shells [3] and the preview-and-confirm design guideline validated with motor-impaired users of an LLM-mediated assistive-robotics interface [5]. First, the command is classified by reversibility: operations that are irreversible — such as file deletion, system configuration changes, or destructive package operations —

trigger a confirmation prompt requiring explicit user approval before execution proceeds. Reversible operations are logged to the undo stack, enabling the user to request rollback at any point through the conversational interface. Second, the approved command is dispatched to a bash subprocess. Standard output and standard error are captured and rendered in the conversational surface as the system's response to the original natural language input.

SynapseOS executes only shell-expressible operations. The system's capability boundary is the Linux command-line toolchain: file management, process control, text processing, package management, network configuration, and any application that exposes a command-line interface. GUI-only interactions — clicking buttons in graphical applications, navigating web page elements, or interacting with visual-only interfaces — are explicitly outside scope, in deliberate contrast to broader desktop agent frameworks that additionally orchestrate GUI automation atop the existing session [16]. This boundary is intentional: it constrains the research to a tractable, verifiable claim that the system can be built, evaluated, and defended within the thesis timeline.

### 2.2 Data Pre-processing

Prior to statistical analysis, all data collected during the benchmark evaluation and user study — not the system itself — undergoes the following processing steps:

- **Task normalization:** task descriptions and expected outcomes are standardized across both the OSWorld benchmark and the custom task suite to ensure consistent evaluation criteria

- **Log parsing and event extraction:** raw session logs from SynapseOS telemetry are parsed to extract per-task metrics — completion time, error events, and undo events
- **Metric computation:** task completion time is computed as the elapsed time between task prompt presentation and task completion event; error rate is computed as the ratio of failed or incorrect task outcomes to total task attempts per participant per condition; SUS and NASA-TLX scores are computed according to their respective standard scoring formulas
- **Participant data anonymization:** all participant records are anonymized at the point of extraction, replacing identifying information with participant ID codes prior to any analysis

### 2.3 Validation

The following validation procedures are applied to ensure the reliability and accuracy of the system and the study instruments:

- **Pilot study:** conducted with five participants prior to the main study to validate the task set difficulty, the questionnaire instruments, and the session procedure; findings from the pilot study are used to revise instruments and procedures before the main data collection begins
- **Intent parsing accuracy:** evaluated using an execution-based scoring methodology [2] — each generated command is executed in a controlled environment and its resulting system state compared against a human-annotated expected outcome for each task utterance, rather than syntax-level string matching, with ambiguous cases resolved through LLM-

assisted functional equivalence assessment at 95% confidence; accuracy is computed as the proportion of first-attempt matches

- **Confirmation policy effectiveness:** evaluated by verifying that all irreversible operations in the task set trigger the confirmation prompt as designed, and that no irreversible operations execute without user approval
- **Undo success rate:** evaluated by executing a representative set of reversible operations and verifying that the undo log correctly restores the prior system state in each case

### 2.4 Study Session Procedure

Each participant session follows a standardized procedure conducted in the following order:

1. **Briefing:** The session facilitator explains the study purpose, session structure, and participant rights. The participant reads and signs the informed consent form. The pre-study screening questionnaire is administered to confirm group classification.
2. **System familiarization:** For the SynapseOS condition, the participant is given a five-minute orientation to the conversational interface and shown how to enter natural language commands. For the native OS condition, no familiarization is provided — participants use their home environment as they normally would. No task-specific training is provided under either condition.
3. **Task completion:** The participant completes the full task set under the assigned condition. Tasks are presented one at a time via a printed task prompt sheet. The session logger

captures start and end times automatically. Screen recording captures all on-screen activity. The participant is instructed to think aloud throughout.

4. **Post-condition questionnaire:** Immediately after completing all tasks under a condition, the participant completes the SUS and NASA-TLX questionnaires for that condition.
5. **Condition changeover:** If the participant has not yet completed both conditions, a short break is offered, the participant moves to the other machine (SynapseOS or native OS), and the session returns to step 2.
6. **Post-study interview:** After both conditions are completed, the facilitator conducts a semi-structured interview of approximately 10–15 minutes covering perceived ease of use, trust in the conversational interface, preference between conditions, and open-ended feedback.
7. **Debrief:** The facilitator discloses the full study objectives, answers participant questions, and thanks the participant. Session data is anonymized immediately upon session close.

Total estimated session duration per participant: 90–120 minutes.

### 2.5 Ethical Considerations

This study involves human participants and is subject to ethics review and approval by the Institutional Review Board (IRB) or equivalent ethics committee of Mapúa University – Makati prior to the commencement of participant recruitment. The following ethical principles are observed throughout the study:

- **Informed consent:** All participants receive a written information sheet describing the study purpose, procedures, data collected, and their rights. Participation is entirely voluntary; participants may withdraw at any time without penalty or consequence.
- **Anonymization:** Participant identities are not recorded. Each participant is assigned an anonymous ID code at the point of enrollment. All collected data — session logs, questionnaire responses, and interview transcripts — are stored and analyzed using participant ID codes only.
- **Data minimization:** Only the data types specified in Section 1.1 are collected. No identifying metadata — names, email addresses, or device identifiers — is retained beyond the consent confirmation step.
- **Data security:** All collected data is stored on password-protected institutional infrastructure accessible only to the research team. Interview recordings are deleted after transcription and verification.
- **Confidentiality:** Study findings are reported in aggregate. No individual participant's data is disclosed in any form that could enable identification.
- **Right to withdraw:** Participants may discontinue the session and request withdrawal of their data at any point during or after the session, up to the point at which anonymization makes individual data unidentifiable.

## Section 3: System Testing

SynapseOS is evaluated against the datasets and study design established in Section 1. The comparison follows a fixed set of metrics and formulas, a statistical analysis plan for testing significance and effect size, and an accounting of the threats to validity this design addresses.

### 3.1 Evaluation Dataset

The system is evaluated against two datasets: the OSWorld benchmark task suite [17] for direct comparison with results in the existing literature, and the custom cross-platform task suite described in Sections 1.1 and 1.6. The user study evaluation involves 20 participants (10 novice, 10 power users); the baseline condition is each participant's primary operating system — Windows 11, macOS, or Linux with GNOME desktop — on a dedicated baseline machine.

### 3.2 Evaluation Profile

The study employs a within-subjects design with two conditions:

- **Condition A:** SynapseOS (conversational interface layer on the Debian 13 evaluation machine)
- **Condition B:** Participant's primary operating system (Windows 11, macOS, or Linux with GNOME desktop, as determined by pre-study screening, on the dedicated baseline machine)

All 20 participants complete both conditions. Condition order is counterbalanced: 10 participants complete Condition A first, and 10 participants complete Condition B first. This counterbalancing controls for order effects and task familiarity.

The two participant groups — novice users and power users — are analyzed both in aggregate and separately to determine whether the effect of the interface condition differs across experience levels. Additionally, primary OS (Windows, macOS, Linux) is recorded as a secondary grouping variable to determine whether results vary across participant backgrounds, and prior conversational AI exposure (frequency of use) is recorded as a covariate to determine whether familiarity with LLM-based tools correlates with SynapseOS performance independent of OS proficiency. The primary analysis is the pooled within-subjects comparison across all 20 participants; per-OS subgroup analyses and AI-exposure correlations are treated as exploratory given the smaller per-group sample sizes expected at n = 20.

### 3.3 Criteria and Formula

The following metrics and formulas are used to evaluate SynapseOS against the baseline:

a) Task Completion Time

Mean task completion time is computed per condition per participant group. Inferential testing and effect size reporting follow the procedures described in Section 3.4.

b) Error Rate

Error rate is computed as the proportion of task attempts resulting in failure or incorrect outcome:

$$Error\ Rate = (Failed\ Tasks + Incorrect\ Outcomes) / Total\ Task\ Attempts$$

Error events are further categorized by type: intent parsing errors (incorrect bash command generated), execution errors (bash subprocess failure), and recovery failures (unsuccessful undo or retry). The same paired statistical test is applied to error rate differences between conditions.

c) User Satisfaction

SUS scores are computed per the standard scoring procedure to produce a score on a 0–100 scale; a score above 68 is considered above-average usability. NASA-TLX scores are computed as the unweighted mean of the six subscale ratings to produce a composite workload score per participant per condition.

Qualitative interview data is analyzed using thematic analysis to identify recurring themes in participant perceptions of both conditions.

d) Statistical Significance and Effect Size

All primary comparisons between conditions are tested at $\alpha = 0.05$. Effect size is reported using Cohen's $d$, with thresholds of 0.2 (small), 0.5 (medium), and 0.8 (large). Minimum sample size justification is based on an a priori power analysis targeting 80% statistical power at the specified significance level and a medium effect size ($d = 0.5$), which yields a minimum of 17 participants for a within-subjects design — satisfied by the 20-participant sample.

## 3.4 Data Analysis Plan

Quantitative analysis proceeds in three stages. First, descriptive statistics — means, standard deviations, and 95% confidence intervals — are computed for all primary metrics (task completion time, error rate, SUS score, NASA-TLX score) per condition and per participant group. Second, the Shapiro-Wilk test is applied to assess normality of each distribution before selecting the appropriate inferential test: the paired-samples t-test for normally distributed data, or the Wilcoxon signed-rank test for non-normally distributed data. Third, effect size (Cohen's $d$) is computed for all statistically significant comparisons.

Subgroup analysis is conducted separately for the novice and power user groups to determine whether the effect of the interface condition differs across populations. The interaction between interface condition and user group is examined to assess whether SynapseOS produces differential effects depending on technical background.

Qualitative data from post-study interviews is analyzed using reflexive thematic analysis following the six-phase procedure described by Braun and Clarke [24]: familiarization with the data, initial code generation, theme search, theme review, theme definition, and write-up. Two members of the research team independently code a random subset of transcripts; inter-rater agreement is assessed and disagreements are resolved through discussion before finalizing the codebook. Themes are reported alongside representative participant quotations.

## 3.5 Threats to Validity

The following threats to validity are identified and the mitigations applied in this study are described.

**Internal validity:**

- **Learning effect:** Participants may perform better on the second condition simply due to task familiarity, independent of interface quality. *Mitigation:* condition order is counterbalanced across participants so that learning effects are distributed equally across both conditions and do not systematically favor either.
- **Demand characteristics:** Participants may behave differently from natural use because they know they are being observed. *Mitigation:* participants are instructed to complete tasks as they naturally would; the specific hypotheses of the study are withheld until the post-study debrief to reduce social desirability bias.
- **Fatigue:** Completing the full task set twice in a single session may degrade performance in the second condition independent of interface quality. *Mitigation:* a short break is offered between conditions and total session length is bounded to approximately 120 minutes.

**External validity:**

- **Sample generalizability:** The study uses a university-recruited sample of 20 participants across three primary OS backgrounds (Windows, macOS, Linux), which limits direct generalization to the broader population of computer users. Findings should be interpreted as indicative rather than definitive; replication with larger and more diverse samples is recommended as a direction for future work.

- **Task generalizability:** The evaluation is bounded by OSWorld coverage and the custom task suite developed for this study. Computing workflows outside this coverage are not evaluated, and results may not extend to all possible use cases.
- **Platform specificity:** SynapseOS is evaluated on a single hardware and software configuration — a dedicated machine running SynapseOS in GUI mode (XFCE desktop environment, X11 session) on a Debian 13 (Trixie) base. Performance characteristics on different hardware, display configurations, or Linux distributions may differ from the reported results.

---

## Notes

1. Terminal User Interface: a text-based interactive interface that runs in a command-line terminal. TUI mode requires no graphical display server or desktop environment and can be accessed remotely over SSH.
2. The SynapseOS prototype uses Qwen2.5-Coder-3B-Instruct, a code-specialized open-weights model selected for its NL2Bash accuracy. See Section 2.1a for selection rationale and benchmark evidence.
3. An inference server loads a language model into memory and processes query requests from the application layer. Ollama is an open-source tool for running language models locally on consumer hardware without cloud connectivity.
4. Open-weights models make their trained parameters publicly available for local download and use; users run inference on their own hardware without routing queries to an external service provider.
5. Quantization reduces the numerical precision of model weights to lower memory requirements. At 4-bit precision, a model that would otherwise require dedicated GPU memory can run on a standard CPU with modest RAM.
6. Low-Rank Adaptation (LoRA) is a parameter-efficient fine-tuning method that introduces a small number of trainable weight matrices while keeping the base model frozen, enabling task-specific adaptation on consumer hardware without full retraining.
7. NL2Bash is a benchmark dataset pairing natural language command descriptions with their corresponding bash shell commands, used to evaluate a model's accuracy in translating natural language into executable commands.

## References

[2] Westenfelder, F., Hemberg, E., Tulla, M., Moskal, S., O'Reilly, U.-M., & Chiricescu, S. (2025). LLM-Supported Natural Language to Bash Translation. *arXiv:2502.06858*. https://arxiv.org/abs/2502.06858

[3] Gyawali, B. R., Achalla, S., Kallas, K., & Kumar, S. (2025). NaSh: Guardrails for an LLM-Powered Natural Language Shell. *arXiv:2506.13028*. https://arxiv.org/abs/2506.13028

[5] Padmanabha, A., Yuan, J., Gupta, J., Karachiwalla, Z., Majidi, C., Admoni, H., & Erickson, Z. (2024). VoicePilot: Harnessing LLMs as Speech Interfaces for Physically Assistive Robots. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology (UIST '24)*. ACM. https://doi.org/10.1145/3654777.3676401

[16] Zhang, C., He, S., Qian, J., Li, B., Li, L., Qin, S., Kang, Y., Ma, M., Liu, G., Lin, Q., Rajmohan, S., Zhang, D., & Zhang, Q. (2025). UFO²: The Desktop AgentOS. *arXiv:2504.14603*. https://arxiv.org/abs/2504.14603

[17] Xie, T., Zhang, D., Chen, J., Li, X., Zhao, S., Cao, R., Hua, T. J., Cheng, Z., Shin, D., Lei, F., Liu, Y., Xu, Y., Zhou, S., Savarese, S., Xiong, C., Zhong, V., & Yu, T. (2024). OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*. https://arxiv.org/abs/2404.07972

[24] Braun, V., & Clarke, V. (2006). Using thematic analysis in psychology. *Qualitative Research in Psychology, 3*(2), 77–101. https://doi.org/10.1191/1478088706qp063oa

[25] Jacobs, J., Lapon, J., & Naessens, V. (2026). Local LLMs for NL2Bash: A Large-Scale Open-Source Model Evaluation for Bash Command Generation. *NDSS Workshop on LLM Assisted Security and Trust Exploration (LAST-X 2026)*. https://www.ndss-symposium.org/wp-content/uploads/lastx2026-49.pdf

## Appendices

This section contains full copies of the study instruments administered to participants, to be inserted once finalized during the pilot validation described in Section 2.3.

### Appendix A: System Usability Scale (SUS)

*[Placeholder — the finalized SUS instrument, as administered to participants after each condition, will be inserted here prior to submission.]*

### Appendix B: NASA Task Load Index (NASA-TLX)

*[Placeholder — the finalized NASA-TLX instrument, as administered to participants after each condition, will be inserted here prior to submission.]*